Universidad Autónoma de Yucatán
Facultad de Matemáticas
Licenciatura en Ingeniería de Software

Programación Estructurada

Propuesta de Proyecto Final

Micha y Micha

Cuentas claras entre roomies

Alumno: Luis Adrian Villanueva Valdivia
Profesor: Emilio Gabriel Rejón Herrera
Modalidad: Individual

Mérida, Yucatán, 21 de octubre de 2026

Contenido

1. Definición inicial de requerimientos del sistema
    1. Antecedentes de la propuesta
    2. Descripción del producto software
    3. Objetivo general y objetivos específicos
    4. Actores del sistema (tipos de usuario)
    5. Diagrama general de casos de uso
    6. Principales interfaces de usuario
2. Mapeo de requerimientos
    1. Requerimientos funcionales
    2. Requerimientos no funcionales
    3. Casos de uso detallados
3. Definición del estándar de codificación
4. Proceso de desarrollo
5. Referencias
6. Anexo: Presentación (10 diapositivas)

1. Definición inicial de requerimientos del sistema

1.1 Antecedentes de la propuesta

Compartir departamento o casa con roomies es muy común entre estudiantes universitarios, sobre todo entre quienes vienen de otras ciudades o estados a estudiar a Mérida. Vivir así reduce el costo de la renta, pero trae consigo una tarea que casi nadie organiza bien: llevar las cuentas de los gastos compartidos.

En un mes típico se pagan la renta, la luz, el agua, el gas, el internet, el súper y cosas de limpieza. Cada gasto lo paga una persona distinta y, muchas veces, no todos participan en todos los gastos (por ejemplo, uno no come en casa y no entra en el súper). La forma habitual de controlarlo es con notas en el celular, mensajes en el grupo de WhatsApp o simplemente "de memoria". El resultado es que nadie sabe con claridad quién le debe a quién ni cuánto, se olvidan pagos pequeños que se acumulan y aparecen discusiones que afectan la convivencia.

Un segundo problema es que, aunque se sepa cuánto ha pagado cada quien, saldar las cuentas suele hacerse con pagos cruzados innecesarios: si hay cinco roomies, cada uno termina transfiriendo a varios de los demás. Esto se puede evitar calculando el saldo neto de cada persona y proponiendo el mínimo razonable de transferencias.

Existen aplicaciones comerciales para dividir gastos (por ejemplo, Splitwise o Tricount), pero dependen de internet y de una cuenta en línea, y algunas limitan funciones en su versión gratuita. Para fines de esta materia, el reto es construir una solución propia, sencilla y local, en lenguaje C, que aplique los temas del curso: arreglos, estructuras, modularidad, apuntadores, condicionales, ciclos y manejo de archivos.

De ahí nace Micha y Micha: un programa de consola que permite a un grupo de roomies registrar sus gastos compartidos, dividirlos de forma justa, consultar el saldo de cada persona y saber exactamente qué transferencias hacer para quedar a mano.

1.2 Descripción del producto software

Micha y Micha es una aplicación de consola, escrita en C (estándar C99), que funciona en una sola computadora y guarda su información en archivos locales. Se usa mediante menús numerados y está pensada para un hogar o grupo de hasta 10 personas.

¿Qué hace?

- **Usuarios y grupos.** Permite registrar a los roomies, crear uno o varios grupos (por ejemplo, "Depa Centro" o "Viaje a Bacalar") y asignar los miembros de cada grupo.

- **Registro de gastos.** Cada gasto guarda su descripción, monto, fecha, categoría (renta, servicios, súper, limpieza, salidas, otros), quién lo pagó y entre quiénes se divide.
- **División del gasto.** El gasto se divide en partes iguales entre los participantes seleccionados o en partes personalizadas (montos específicos por persona). El sistema valida que la suma de las partes sea igual al total.
- **Saldos.** Calcula el saldo neto de cada persona: lo que pagó menos lo que le correspondía, considerando también los pagos (liquidaciones) ya realizados. Un saldo positivo indica que le deben; uno negativo, que debe.
- **Simplificación de deudas.** Con un algoritmo *greedy* empareja repetidamente al mayor deudor con el mayor acreedor y genera la lista de transferencias sugeridas. Con n personas se requieren como máximo $n - 1$ transferencias, en lugar de pagos cruzados entre todos.
- **Liquidaciones.** Registra los pagos entre roomies ("Ana le pagó $350 a Luis") y actualiza los saldos.
- **Reportes.** Muestra el historial de gastos, el total gastado por categoría y por persona, y filtra por mes.
- **Persistencia.** Guarda y carga toda la información en archivos, para que no se pierda al cerrar el programa.

¿Cómo se relaciona con la materia?

Tema del curso	Uso en Micha y Micha
Estructuras (`struct`)	Cada entidad del sistema se modela con un `struct`: `Usuario`, `Grupo`, `MiembroGrupo`, `Categoria`, `Gasto`, `DivisionGasto` y `Liquidacion` (equivalentes a las "tablas" del diseño).
Arreglos	Arreglos de estructuras con tamaño máximo (`MAX_USUARIOS`, `MAX_GASTOS`, etc.) y arreglos de saldos por persona que se ordenan en la simplificación de deudas.
Modularidad (funciones)	Cada tarea se resuelve en su propia función con parámetros y valor de retorno (por ejemplo, `registrarGasto()`, `calcularSaldos()`, `simplificarDeudas()`), y el `main` solo llama a las funciones desde el menú. Las funciones relacionadas se agrupan en archivos `.c/.h`: usuarios, grupos, gastos, saldos, reportes, archivos y utilidades.
Apuntadores	Paso por referencia de arreglos y estructuras a las funciones (por ejemplo, `registrarGasto(Gasto *gastos, int *numGastos, ...)`) para modificar datos sin copiarlos.
Condicionales y ciclos	Menús con `switch`, validación de entradas, recorridos de arreglos y el ciclo principal del algoritmo greedy.

Archivos	Lectura y escritura de los datos en archivos de texto (`fopen`, `fprintf`, `fscanf`).

Modelo de datos

Las "tablas" planteadas en la idea original se implementan como estructuras guardadas en arreglos y en un archivo por cada una:

Estructura	Campos principales
`Usuario`	id, nombre, PIN de 4 dígitos, esAdministrador
`Grupo`	id, nombre, idAdministrador
`MiembroGrupo`	idGrupo, idUsuario
`Categoria`	id, nombre
`Gasto`	id, idGrupo, idPagador, idCategoria, descripción, montoCentavos, fecha, tipoDivision (igual / personalizada)
`DivisionGasto`	idGasto, idUsuario, montoCentavos (la parte que le toca a cada persona)
`Liquidacion`	id, idGrupo, idDeudor, idAcreedor, montoCentavos, fecha

Los montos se guardan en centavos como enteros (`long`) para evitar errores de redondeo de los números de punto flotante; al mostrarse se convierten a pesos con dos decimales.

Algoritmo de simplificación de deudas

Idea: primero se calcula el saldo neto de cada miembro (lo pagado − lo que le correspondía ± liquidaciones). La suma de todos los saldos siempre es cero. Después:

1. Se separa a las personas en deudores (saldo < 0) y acreedores (saldo > 0).
2. Se toma al mayor deudor y al mayor acreedor.
3. El deudor transfiere al acreedor el menor de los dos montos (|deuda| o crédito).
4. Se actualizan ambos saldos; quien quede en cero sale de la lista.
5. Se repite hasta que todos los saldos sean cero.

En cada paso al menos una persona queda en cero (y en el último quedan dos), por lo que con *n* personas se generan como máximo *n* − 1 transferencias.

Ejemplo: saldos Luis +$600, Ana −$400, Beto −$150, Caro −$50. Paso 1: Ana → Luis $400 (Luis queda en +$200). Paso 2: Beto → Luis $150 (Luis +$50). Paso 3: Caro → Luis $50. Tres transferencias para cuatro personas.

1.3 Objetivo general y objetivos específicos

Objetivo general

Desarrollar un programa de consola en lenguaje C, aplicando los principios de la programación estructurada, que permita a un grupo de roomies registrar y dividir sus gastos compartidos, conocer en todo momento el saldo de cada integrante y saldar sus deudas con el menor número razonable de transferencias.

Objetivos específicos

1. Diseñar las estructuras de datos (`struct` y arreglos) que representen usuarios, grupos, categorías, gastos, divisiones y liquidaciones.
2. Implementar el registro de gastos con división en partes iguales o personalizadas, validando que la suma de las partes coincida con el total.
3. Calcular el saldo neto de cada integrante del grupo considerando gastos y liquidaciones registradas.
4. Implementar un algoritmo greedy de simplificación de deudas que sugiera, como máximo, $n - 1$ transferencias para n integrantes.
5. Generar reportes de gastos por categoría, por persona y por mes.
6. Guardar y recuperar la información en archivos para que los datos persistan entre ejecuciones.
7. Organizar el código en módulos independientes (`.c/.h`) que usen apuntadores para el paso de datos por referencia, siguiendo un estándar de codificación definido.

1.4 Actores del sistema (tipos de usuario)

Actor	Descripción	Funciones permitidas
Roomie (actor principal)	Integrante de un grupo que comparte gastos. Es quien usa el sistema día a día para anotar lo que paga y revisar cuánto debe o le deben. Se identifica con su nombre y un PIN de 4 dígitos.	Iniciar sesión, registrar gastos, consultar saldos, ver la simplificación de deudas, registrar liquidaciones y generar reportes de los grupos a los que pertenece.
Administrador del hogar	Roomie que, además, se encarga de configurar el sistema: normalmente quien crea el grupo. Es una especialización del Roomie: puede hacer todo lo que hace un Roomie y tiene permisos extra.	Todo lo del Roomie, más: crear, editar y eliminar grupos; dar de alta y de baja miembros; administrar las categorías de gasto.

1.5 Diagrama general de casos de uso

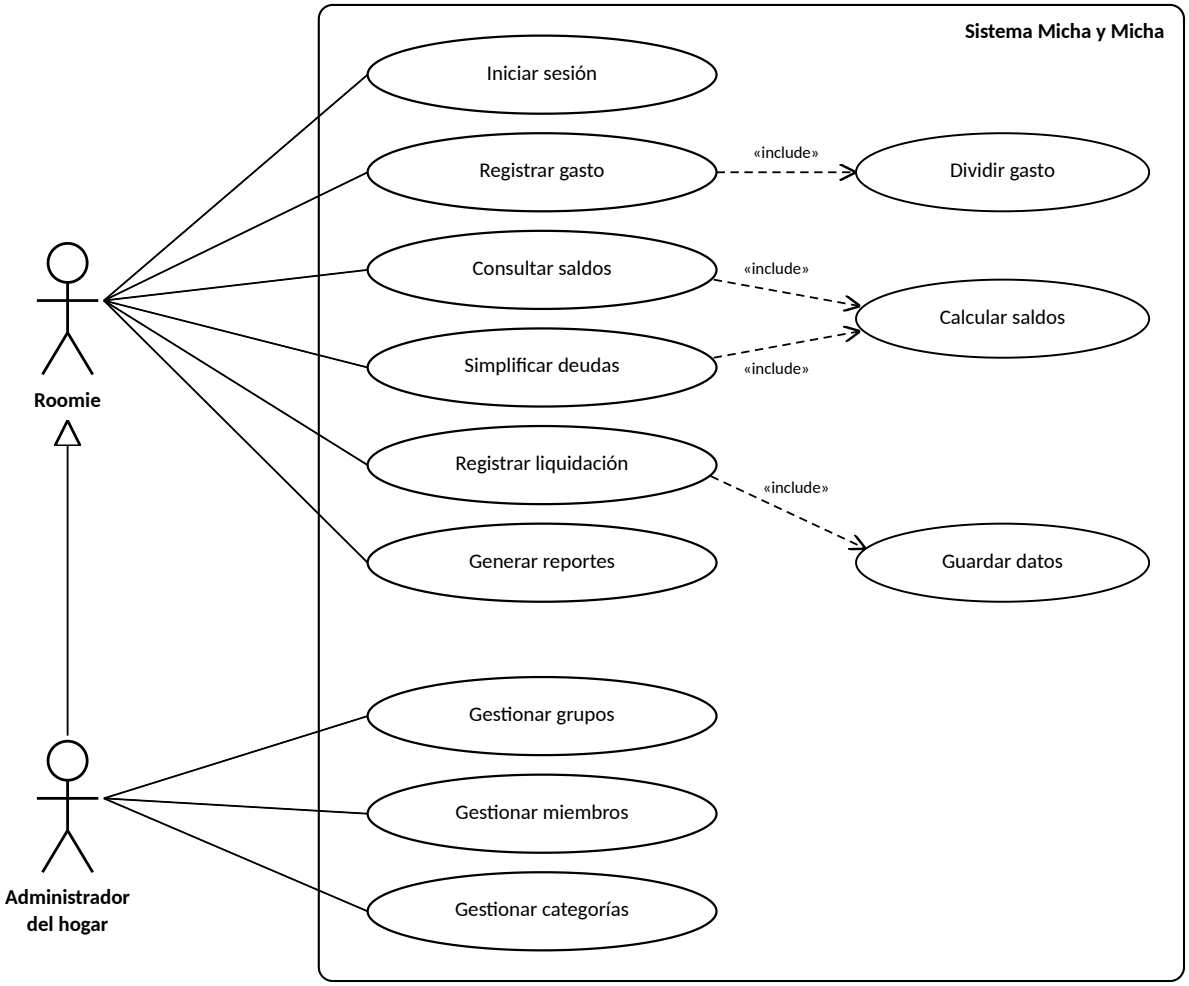


Figura 1. Diagrama general de casos de uso. El Administrador del hogar hereda (generalización) los casos de uso del Roomie.

1.6 Principales interfaces de usuario

La interfaz es de consola, con menús numerados. Estos son bocetos de las pantallas principales:

Pantalla de inicio de sesión

```
==============================================
            M I C H A   Y   M I C H A
        Cuentas claras entre roomies
==============================================
 Usuarios registrados:
   1) Luis    2) Ana    3) Beto    4) Caro
----------------------------------------------
 Selecciona tu usuario: 1
 PIN (4 digitos): 4821
 Bienvenido, Luis. Grupo activo: Depa Centro
```

Menú principal

```
=============== MENU PRINCIPAL ===============
 Grupo: Depa Centro (4 miembros)   [Admin]
----------------------------------------------
  1. Registrar gasto
  2. Consultar saldos
  3. Simplificar deudas
  4. Registrar liquidacion (pago)
  5. Reportes
  6. Cambiar de grupo
  7. Administracion (grupos/miembros/categorias)
  0. Guardar y salir
----------------------------------------------
 Opcion: _
```

Registrar gasto

```
============== REGISTRAR GASTO ===============
 Descripcion: Super de la semana
 Monto ($): 840.00
 Fecha (dd/mm/aaaa): 21/10/2026
 Categoria: 1)Renta 2)Servicios 3)Super 4)Limpieza 5)Otros
   -> 3
 Quien pago?  1)Luis 2)Ana 3)Beto 4)Caro  -> 1
 Participantes (ej. 1,2,4): 1,2,4
 Division: 1) Partes iguales  2) Personalizada -> 1
----------------------------------------------
 Luis $280.00 | Ana $280.00 | Caro $280.00
 Gasto registrado correctamente.
```

Saldos y simplificación de deudas

```
============ SALDOS: Depa Centro =============
  Luis     +$600.00   (le deben)
  Ana      -$400.00   (debe)
  Beto     -$150.00   (debe)
  Caro      -$50.00   (debe)
============ TRANSFERENCIAS SUGERIDAS ========
  1. Ana  -> Luis   $400.00
  2. Beto -> Luis   $150.00
  3. Caro -> Luis    $50.00
 Total: 3 transferencias (maximo n-1 = 3)
```

Reporte por categoría

```
====== REPORTE OCTUBRE 2026: Depa Centro =====
  Categoria        Total        %
  Renta         $8,000.00    71.4
  Servicios     $1,450.00    12.9
  Super         $1,350.00    12.1
  Limpieza        $400.00     3.6
---------------------------------------------
  TOTAL        $11,200.00   100.0
```

2. Mapeo de requerimientos

Siguiendo la distinción de las fuentes del curso, los requerimientos funcionales describen las interacciones del sistema con su ambiente (qué hace), de forma independiente a la implementación; los no funcionales describen restricciones y atributos de calidad visibles para el usuario (cómo lo hace), tomando como referencia los atributos de calidad de la norma ISO/IEC 9126.

2.1 Requerimientos funcionales

ID	Nombre	Descripción	Prioridad
RF-01	Iniciar sesión	El sistema permitirá a un usuario identificarse seleccionando su nombre e ingresando su PIN de 4 dígitos, y mostrará el menú según su tipo (Roomie o Administrador).	Alta
RF-02	Registrar usuarios	El sistema permitirá registrar nuevos usuarios con nombre y PIN, sin repetir nombres.	Alta
RF-03	Gestionar grupos	El Administrador podrá crear, renombrar y eliminar grupos de gastos.	Alta
RF-04	Gestionar miembros	El Administrador podrá agregar y quitar miembros de un grupo. No se podrá quitar a un miembro con saldo distinto de cero.	Alta
RF-05	Gestionar categorías	El Administrador podrá agregar, renombrar y eliminar categorías de gasto. El sistema incluirá categorías predeterminadas (Renta, Servicios, Súper, Limpieza, Otros).	Media
RF-06	Registrar gasto	El sistema permitirá registrar un gasto con descripción, monto, fecha, categoría, pagador y participantes.	Alta
RF-07	Dividir en partes iguales	El sistema dividirá el monto entre los participantes seleccionados; si la división no es exacta en centavos, el residuo se asignará al pagador.	Alta
RF-08	Dividir en partes personalizadas	El sistema permitirá capturar la parte de cada participante y validará que la suma sea igual al total del gasto.	Alta
RF-09	Eliminar o corregir gasto	El pagador o el Administrador podrán eliminar un gasto registrado por error, y los saldos se recalcularán.	Media

RF-10	Consultar saldos	El sistema mostrará el saldo neto de cada miembro del grupo, indicando si debe o si le deben.	Alta
RF-11	Simplificar deudas	El sistema calculará y mostrará la lista de transferencias sugeridas mediante el algoritmo greedy, con un máximo de $n - 1$ transferencias.	Alta
RF-12	Registrar liquidación	El sistema permitirá registrar un pago entre dos miembros (deudor, acreedor, monto, fecha) y actualizará los saldos.	Alta
RF-13	Historial de gastos	El sistema mostrará la lista de gastos y liquidaciones del grupo ordenada por fecha.	Media
RF-14	Reporte por categoría	El sistema mostrará el total gastado por categoría y su porcentaje, con filtro por mes.	Media
RF-15	Reporte por persona	El sistema mostrará cuánto ha pagado y cuánto le ha correspondido a cada miembro.	Media
RF-16	Guardar datos	El sistema guardará la información en archivos al registrar cambios y al salir.	Alta
RF-17	Cargar datos	Al iniciar, el sistema cargará la información de los archivos; si no existen, los creará vacíos con las categorías predeterminadas.	Alta

2.2 Requerimientos no funcionales

ID	Atributo (ISO 9126)	Descripción
RNF-01	Portabilidad	El programa estará escrito en C estándar (C99), usando solo bibliotecas estándar (`stdio.h`, `stdlib.h`, `string.h`, `ctype.h`), y compilará sin errores con `gcc` en Windows (MinGW) y Linux.
RNF-02	Usabilidad	La interfaz será de consola en español, con menús numerados de máximo 9 opciones y la opción 0 para regresar o salir. Cada pantalla indicará qué dato se espera.
RNF-03	Confiabilidad (tolerancia a fallos)	El sistema validará todas las entradas (tipo, rango y formato de fecha). Una entrada inválida mostrará un mensaje claro y volverá a pedir el dato sin cerrar el programa.

RNF-04	Funcionalidad (precisión)	Los montos se manejarán en centavos como enteros para evitar errores de redondeo; la suma de saldos de un grupo siempre será exactamente cero.
RNF-05	Eficiencia	Cualquier operación (registrar, calcular saldos, simplificar deudas, generar reportes) responderá en menos de 1 segundo con la capacidad máxima del sistema.
RNF-06	Capacidad	El sistema soportará hasta 10 miembros por grupo, 5 grupos, 20 categorías y 500 gastos, definidos mediante macros para poder ajustarse.
RNF-07	Confiabilidad (recuperación)	Los datos se guardarán en archivos de texto; si un archivo falta o está dañado, el sistema avisará y continuará con datos vacíos en lugar de terminar abruptamente.
RNF-08	Seguridad básica	Cada usuario tendrá un PIN numérico de 4 dígitos para entrar al sistema y el PIN nunca se mostrará en listados ni reportes. Solo el Administrador podrá acceder a las opciones de administración.
RNF-09	Facilidad de mantenimiento	El código estará dividido en módulos `.c/.h` con una responsabilidad cada uno y seguirá el estándar de codificación de la sección 3.

2.3 Casos de uso detallados

Cada caso de uso describe su flujo principal y sus rutas alternativas (excepciones), siguiendo el formato del documento de referencia del curso.

CU-01 Iniciar sesión

Actores	Roomie, Administrador del hogar
Descripción	El usuario se identifica para entrar al sistema.
Precondiciones	El sistema cargó los datos de los archivos. Existe al menos un usuario registrado.
Flujo principal	1. El sistema muestra la lista de usuarios. 2. El usuario selecciona su número. 3. El sistema solicita el PIN. 4. El usuario ingresa su PIN. 5. El sistema valida el PIN y muestra el menú principal según el tipo de usuario.
Rutas alternativas	**E1** Número de usuario inexistente: el sistema muestra un error y vuelve al paso 1. **E2** PIN incorrecto: el sistema avisa y permite reintentar; tras 3 intentos fallidos regresa a la lista de usuarios. **E3** No hay usuarios: el sistema pide registrar al primer usuario, que queda como Administrador.
Postcondiciones	El usuario está dentro del sistema con su grupo activo.

CU-02 Registrar gasto (incluye CU-03 Dividir gasto)

Actores	Roomie
Descripción	El usuario registra un gasto compartido del grupo activo.
Precondiciones	El usuario inició sesión y pertenece al grupo activo, que tiene al menos 2 miembros.

Flujo principal	1. El usuario elige "Registrar gasto". 2. El sistema solicita descripción, monto, fecha y categoría. 3. El usuario captura los datos. 4. El sistema solicita quién pagó y los participantes. 5. El usuario los selecciona. 6. El sistema pregunta el tipo de división y ejecuta **CU-03 Dividir gasto**. 7. El sistema muestra el resumen y pide confirmación. 8. El usuario confirma. 9. El sistema guarda el gasto y sus divisiones (incluye **Guardar datos**).
Rutas alternativas	**E1** Monto no numérico, cero o negativo: el sistema muestra un error y lo vuelve a pedir. **E2** Fecha con formato inválido o inexistente: el sistema la vuelve a pedir. **E3** Participante que no pertenece al grupo: el sistema lo indica y repite la selección. **E4** El usuario cancela en el paso 8: el gasto no se guarda y se regresa al menú. **E5** Se alcanzó el máximo de gastos: el sistema avisa y no permite registrar más.
Postcondiciones	El gasto queda registrado y los saldos del grupo se actualizan.

CU-03 Dividir gasto

Actores	Roomie (incluido desde CU-02)
Flujo principal	**Partes iguales:** 1. El sistema divide el monto en centavos entre el número de participantes. 2. Si hay residuo, se lo asigna al pagador. 3. Muestra la parte de cada uno. **Personalizada:** 1. El sistema pide la parte de cada participante. 2. El usuario las captura. 3. El sistema verifica que la suma sea igual al total.
Rutas alternativas	**E1** La suma de las partes personalizadas no coincide con el total: el sistema muestra la diferencia y pide corregir las partes. **E2** Parte negativa: el sistema la rechaza y la vuelve a pedir.
Postcondiciones	Cada participante tiene asignada su parte del gasto.

CU-04 Consultar saldos (incluye CU-05 Calcular saldos)

Actores	Roomie
Precondiciones	El usuario inició sesión y tiene un grupo activo.

Flujo principal	1. El usuario elige "Consultar saldos". 2. El sistema ejecuta **CU-05 Calcular saldos**: para cada miembro suma lo pagado, resta lo que le correspondía y ajusta las liquidaciones. 3. El sistema muestra el saldo de cada miembro indicando "debe" o "le deben".
Rutas alternativas	**E1** El grupo no tiene gastos: el sistema muestra "No hay gastos registrados" y todos los saldos en $0.00.
Postcondiciones	El usuario conoce el estado de cuentas del grupo.

CU-06 Simplificar deudas

Actores	Roomie
Precondiciones	El usuario inició sesión y tiene un grupo activo.
Flujo principal	1. El usuario elige "Simplificar deudas". 2. El sistema ejecuta **CU-05 Calcular saldos**. 3. El sistema separa deudores y acreedores. 4. Mientras existan saldos distintos de cero: toma al mayor deudor y al mayor acreedor, genera una transferencia por el menor de los dos montos y actualiza los saldos. 5. El sistema muestra la lista de transferencias y el total.
Rutas alternativas	**E1** Todos los saldos están en cero: el sistema muestra "El grupo está a mano, no hay deudas".
Postcondiciones	Se muestran como máximo $n - 1$ transferencias sugeridas. Los datos no se modifican hasta que se registren liquidaciones.

CU-07 Registrar liquidación

Actores	Roomie
Precondiciones	El usuario inició sesión; existen al menos dos miembros en el grupo.
Flujo principal	1. El usuario elige "Registrar liquidación". 2. El sistema puede mostrar las transferencias sugeridas para elegir una, o permitir capturarla manualmente. 3. El usuario indica deudor, acreedor, monto y fecha. 4. El sistema pide confirmación. 5. El usuario confirma y el sistema guarda la liquidación (incluye **Guardar datos**).

Rutas alternativas	**E1** Deudor y acreedor son la misma persona: el sistema lo rechaza. **E2** El monto supera la deuda del deudor: el sistema advierte y pide confirmar o corregir. **E3** El usuario cancela: no se guarda nada.
Postcondiciones	La liquidación queda registrada y los saldos se actualizan.

CU-08 Generar reportes

Actores	Roomie
Flujo principal	1. El usuario elige "Reportes". 2. El sistema ofrece: historial, por categoría o por persona. 3. El usuario elige el tipo y, opcionalmente, un mes (mm/aaaa). 4. El sistema calcula los totales y muestra el reporte en forma de tabla.
Rutas alternativas	**E1** No hay gastos en el periodo: el sistema lo indica. **E2** Mes con formato inválido: el sistema lo vuelve a pedir.
Postcondiciones	El usuario visualiza el reporte solicitado.

CU-09 Gestionar grupos

Actores	Administrador del hogar
Flujo principal	1. El Administrador entra a "Administración > Grupos". 2. El sistema muestra los grupos y las opciones Crear, Renombrar y Eliminar. 3. El Administrador elige una opción y captura los datos. 4. El sistema aplica el cambio y guarda.
Rutas alternativas	**E1** Nombre vacío o repetido: el sistema lo rechaza. **E2** Eliminar un grupo con saldos pendientes: el sistema no lo permite hasta que todos estén en cero. **E3** Se alcanzó el máximo de grupos: el sistema avisa.
Postcondiciones	Los grupos quedan actualizados.

CU-10 Gestionar miembros

Actores	Administrador del hogar
Flujo principal	1. El Administrador entra a "Administración > Miembros". 2. El sistema muestra los miembros del grupo activo. 3. El Administrador elige Agregar (usuario existente o nuevo) o Quitar. 4. El sistema aplica el cambio y guarda.

Rutas alternativas	**E1** El usuario ya es miembro: el sistema lo indica. **E2** Quitar a un miembro con saldo distinto de cero: el sistema no lo permite. **E3** El grupo llegó a 10 miembros: el sistema avisa.
Postcondiciones	La lista de miembros queda actualizada.

CU-11 Gestionar categorías

Actores	Administrador del hogar
Flujo principal	1. El Administrador entra a "Administración > Categorías". 2. El sistema muestra las categorías. 3. El Administrador agrega, renombra o elimina una. 4. El sistema aplica el cambio y guarda.
Rutas alternativas	**E1** Nombre repetido o vacío: el sistema lo rechaza. **E2** Eliminar una categoría con gastos asociados: el sistema no lo permite.
Postcondiciones	Las categorías quedan actualizadas.

CU-12 Guardar datos (incluido)

Actores	Sistema (incluido desde los casos que modifican datos y al salir)
Flujo principal	1. El sistema abre cada archivo de datos en modo escritura. 2. Escribe todos los registros del arreglo correspondiente. 3. Cierra los archivos.
Rutas alternativas	**E1** No se puede abrir o escribir el archivo: el sistema muestra un aviso, conserva los datos en memoria y permite reintentar.
Postcondiciones	La información queda almacenada en disco.

3. Definición del estándar de codificación

El estándar se aplicará a todo el código del proyecto. Se basa en las convenciones más usadas en C y en la definición de *identificador* del manual de estándares del curso ("iniciando con mayúscula cada palabra significativa, cuando el lenguaje lo reconozca"), con el fin de favorecer la facilidad de mantenimiento y la facilidad de análisis que pide la norma ISO/IEC 9126.

3.1 Nombrado

Elemento	Convención	Ejemplos
Variables	`camelCase`, nombres descriptivos en español; sin abreviaturas ambiguas.	`montoCentavos`, `numGastos`, `idPagador`
Contadores de ciclo	Una letra solo en ciclos cortos.	`i`, `j`
Funciones	`camelCase` empezando con verbo en infinitivo.	`registrarGasto()`, `calcularSaldos()`, `simplificarDeudas()`
Estructuras (`typedef struct`)	`PascalCase`, sustantivo en singular.	`Usuario`, `Gasto`, `DivisionGasto`
Macros y constantes	`MAYUSCULAS_CON_GUION_BAJO`.	`MAX_MIEMBROS`, `LONG_NOMBRE`, `ARCHIVO_GASTOS`
Enumeraciones	Tipo en `PascalCase`, valores en mayúsculas.	`TipoDivision { DIVISION_IGUAL, DIVISION_PERSONALIZADA }`
Apuntadores	El `*` junto al nombre; prefijo `p` solo si aclara.	`Gasto *gastos`, `int *pNumGastos`
Archivos	Minúsculas, un módulo por par `.c/.h`.	`gastos.c`, `gastos.h`, `main.c`
Guardas de encabezado	`MICHAYMICHA_MODULO_H`.	`#ifndef MICHAYMICHA_GASTOS_H`

3.2 Organización de archivos y bibliotecas

```
michaymicha/
├── main.c            Punto de entrada y menú principal
├── estructuras.h     typedef struct y macros globales (MAX_*)
├── usuarios.c/.h     Alta de usuarios e inicio de sesión
├── grupos.c/.h       Grupos y miembros
├── gastos.c/.h       Registro, división y eliminación de gastos
├── saldos.c/.h       Cálculo de saldos y simplificación de deudas
├── reportes.c/.h     Historial y reportes
├── archivos.c/.h     Guardar y cargar datos
├── utilidades.c/.h   Lectura validada de entradas, fechas, formato de montos
└── datos/            usuarios.txt, grupos.txt, gastos.txt, ...
```

- Solo bibliotecas estándar: `stdio.h`, `stdlib.h`, `string.h`, `ctype.h`. No se usarán `conio.h` ni bibliotecas exclusivas de un sistema operativo, para mantener la portabilidad (RNF-01).
- Cada `.h` contiene únicamente prototipos, macros y tipos; la implementación va en su `.c`.
- No se usarán variables globales; los datos se pasan por apuntador a las funciones.
- Compilación: `gcc -std=c99 -Wall -Wextra -o michaymicha *.c`. El código debe compilar sin advertencias.

3.3 Formato

- Sangría de 4 espacios (sin tabuladores) y líneas de máximo 100 caracteres.
- Llaves estilo K&R: la llave de apertura en la misma línea de la instrucción; siempre se usan llaves, incluso en `if` de una línea.
- Un espacio alrededor de operadores y después de comas; una línea en blanco entre funciones.
- Una sola declaración de variable por línea, inicializada al declararse.
- Funciones cortas (idealmente menos de 40 líneas) con una sola responsabilidad.

3.4 Comentarios

Bloque de encabezado de archivo (al inicio de cada `.c` y `.h`):

```
/*
 * Archivo:     saldos.c
 * Proyecto:    Micha y Micha
 * Autor:       Luis Adrian Villanueva Valdivia
 * Fecha:       28/10/2026
 * Version:     1.0
 * Descripcion: Calculo de saldos por miembro y simplificacion
 *              de deudas mediante un algoritmo greedy.
 */
```

Bloque de función (antes de cada función, en el `.c`):

```
/*
 * Funcion:     simplificarDeudas
 * Descripcion: Genera la lista minima de transferencias emparejando
 *              al mayor deudor con el mayor acreedor.
 * Parametros:  saldos         - arreglo de saldos en centavos (entrada)
 *              numMiembros    - cantidad de elementos del arreglo
 *              transferencias - arreglo donde se escriben los resultados
 * Retorno:     Numero de transferencias generadas (maximo numMiembros - 1).
 * Precondic.:  La suma de los saldos es cero.
 */
int simplificarDeudas(const long saldos[], int numMiembros,
                      Transferencia *transferencias) {
    ...
}
```

- **Comentarios en línea** solo para explicar el *porqué* de una decisión no evidente (por ejemplo, "el residuo se asigna al pagador para que la suma cuadre"), no para repetir lo que hace el código.
- Cada `struct` lleva un comentario breve por campo.
- Se eliminará el código comentado o de prueba antes de cada entrega.

3.5 Justificación

Se eligió `camelCase` para variables y funciones y `PascalCase` para tipos porque distinguen visualmente qué es un dato, qué es una acción y qué es un tipo, y coinciden con la definición de identificador del manual de estándares. Las mayúsculas para macros son la convención universal en C y evitan confundirlas con variables. Los bloques de comentarios estandarizados documentan el propósito, los parámetros y las precondiciones de cada función, lo que facilita revisar, probar y modificar el código (atributos de *facilidad de análisis* y *facilidad de cambio* de ISO/IEC 9126). Por último, limitarse a C99 y a bibliotecas estándar garantiza que el programa compile igual en cualquier computadora del laboratorio (*portabilidad*).

4. Proceso de desarrollo

Se usará un modelo incremental, en línea con los procesos principales del ciclo de vida de ISO/IEC 12207 (requisitos, diseño, construcción, pruebas y entrega). Cada incremento produce una versión del programa que compila y funciona, y agrega un conjunto de requerimientos. Así siempre existe una versión entregable y los errores se detectan temprano.

4.1 Incrementos planeados

Incremento	Periodo	Contenido	Requerimientos
0. Propuesta	15 – 21 oct	Análisis, requerimientos, casos de uso, estándar y plan.	Documento
1. Base	22 – 26 oct	Estructuras, macros, utilidades de entrada validada, guardar/cargar archivos, usuarios e inicio de sesión.	RF-01, 02, 16, 17
2. Grupos	27 – 30 oct	Grupos, miembros, categorías y menú de administración.	RF-03, 04, 05
3. Gastos	31 oct – 3 nov	Registro de gastos, división igual y personalizada, eliminación.	RF-06 a RF-09
4. Saldos	4 – 7 nov	Cálculo de saldos, simplificación de deudas y liquidaciones.	RF-10, 11, 12
5. Cierre	8 – 11 nov	Reportes, pruebas completas, corrección de errores, manual de usuario y entrega final.	RF-13, 14, 15

Entrega final del proyecto: 11 de noviembre de 2026. Cada incremento dura de 4 a 5 días y termina con una versión que compila y funciona.

4.2 Herramientas

Herramienta	Uso
Visual Studio Code	Editor de código, con extensión de C/C++ para resaltado y depuración.
GCC (MinGW en Windows)	Compilación con `-std=c99 -Wall -Wextra`.
GDB	Depuración de errores de lógica y de apuntadores.

Git + GitHub	Control de versiones: un commit por avance funcional, con mensajes descriptivos ("Agrega división personalizada de gastos"). Una etiqueta (*tag*) por incremento: `v0.1`, `v0.2`, …
GitHub Projects (tablero Kanban)	Organización de tareas en columnas Pendiente / En progreso / Hecho; cada tarea se liga a un requerimiento (RF/RNF).
Hoja de cálculo	Bitácora de trabajo y casos de prueba.

4.3 Organización y monitoreo

Al ser un proyecto individual, el alumno cubre todos los roles (análisis, diseño, programación y pruebas). Para no perder el control del avance:

- **Planeación por incremento:** al inicio de cada incremento se pasan al tablero las tareas de los requerimientos que le corresponden, con una estimación en horas.
- **Revisión por incremento:** al cierre de cada incremento se verifica que el programa compile sin advertencias, se ejecutan los casos de prueba del incremento y se compara lo planeado contra lo terminado.
- **Pruebas:** por cada caso de uso se definen casos de prueba con entradas válidas e inválidas (las rutas alternativas E1, E2, …). Para el algoritmo de simplificación se comprueba que la suma de saldos sea cero y que el número de transferencias no exceda $n - 1$.

4.4 Bitácora

Se llevará una bitácora con un registro por cada sesión de trabajo, con este formato:

Fecha	Inicio – fin	Horas	Actividad / tarea	Req.	Problemas encontrados	Solución
23/10/2026	18:00 – 20:00	2.0	Definir structs y macros en `estructuras.h`	RNF-06	Elegir cómo guardar montos	Usar centavos con `long`

4.5 Medición del trabajo

- **Horas invertidas** por incremento y por tipo de actividad (análisis, programación, pruebas, documentación), obtenidas de la bitácora.
- **Avance:** porcentaje de requerimientos funcionales terminados y probados respecto al total (17).

- **Desviación:** horas estimadas contra horas reales por tarea, para mejorar las estimaciones siguientes.
- **Calidad:** número de casos de prueba aprobados y de errores encontrados y corregidos por incremento.
- **Actividad en el repositorio:** número de commits por incremento como evidencia de trabajo constante.

Aunque el proyecto es individual, el mismo esquema (tablero, bitácora por persona y métricas) permitiría medir el trabajo de cada integrante y del grupo si el proyecto se hiciera en equipo.

5. Referencias

- Análisis de requerimientos para un sistema de tratamiento de diabetes mediante dispositivos móviles (Cap. 3). Material de apoyo del curso.
- Requerimientos funcionales y no funcionales. Material de apoyo del curso.
- Requisitos de Software. Fundamentos de Ingeniería de Software. Material de apoyo del curso.
- Estándares para el Desarrollo de Sistemas Informáticos (Manual de estándares). Material de apoyo del curso.
- Los Estándares de Calidad ISO para Desarrollo de Software (ISO 9001, ISO 9000-3, ISO/IEC 9126). Material de apoyo del curso.
- Normas y Estándares de calidad para el desarrollo de Software (ISO/IEC 12207, ISO/IEC 9126). Material de apoyo del curso.

PROGRAMACIÓN ESTRUCTURADA · PROPUESTA DE PROYECTO FINAL

Micha y Micha

Cuentas claras entre roomies: un programa en C para registrar, dividir y saldar gastos compartidos.

> **Actividad:** Propuesta de proyecto individual — definición inicial de requerimientos del sistema: antecedentes, descripción del producto software, objetivos, diagrama de casos de uso y actores del sistema.

Luis Adrian Villanueva Valdivia
Licenciatura en Ingeniería de Software · Facultad de Matemáticas, UADY

Prof. Emilio Gabriel Rejón Herrera
21 de octubre de 2026

ANTECEDENTES · EL PROBLEMA

En un depa de roomies, nadie sabe bien quién le debe a quién

- Renta, luz, agua, gas, internet, súper… cada gasto lo paga alguien distinto.
- No todos participan en todos los gastos.
- Las cuentas se llevan en notas del celular, en el grupo de WhatsApp o "de memoria".
- Resultado: pagos olvidados, deudas acumuladas y discusiones que afectan la convivencia.

Situación común en estudiantes

Muchos estudiantes que llegan a Mérida de otras ciudades comparten casa para reducir la renta. Vivir así abarata la renta, pero nadie organiza bien las cuentas.

Saldar cuentas genera pagos cruzados innecesarios

6 pagos posibles entre 4 roomies si cada pareja salda por separado

3 transferencias bastan si se calcula el saldo neto de cada persona ($n - 1$)

Alternativas existentes

Apps como Splitwise o Tricount dependen de internet y de una cuenta en línea, y algunas limitan funciones gratuitas.

La oportunidad

Construir una herramienta propia, local y sencilla en **lenguaje C** que aplique los temas del curso: arreglos, estructuras, módulos, apuntadores, condicionales y archivos.

DESCRIPCIÓN DEL PRODUCTO SOFTWARE

Micha y Micha: aplicación de consola en C para gastos compartidos

- Programa en **C99**, con menús numerados en español.
- Funciona en una sola computadora; los datos se guardan en **archivos**.
- Grupos de hasta **10 personas** (depa, viaje, etc.).
- Montos en **centavos enteros** para evitar errores de redondeo.

Modelo de datos (structs + arreglos)

`Usuario` · `Grupo` · `MiembroGrupo` · `Categoria` · `Gasto` · `DivisionGasto` · `Liquidacion`

Módulos

usuarios · grupos · gastos · saldos · reportes · archivos · utilidades

Qué hace y qué lo hace destacar

- Crear grupos y registrar miembros
- Registrar gastos por categoría
- Dividir en partes **iguales** o **personalizadas**
- Ver el **saldo** de cada persona
- Registrar pagos (liquidaciones)
- Reportes por categoría, persona y mes

Simplificación de deudas (greedy)

Empareja al mayor deudor con el mayor acreedor hasta que todos quedan en cero: **máximo *n* − 1 transferencias**.

+600	−400	−150	−50
Luis	Ana	Beto	Caro

Ana → Luis $400 · Beto → Luis $150 · Caro → Luis $50 = **3 transferencias**

OBJETIVO GENERAL

¿Qué busca el sistema?

> Desarrollar un programa de consola en lenguaje C, aplicando los principios de la programación estructurada, que permita a un grupo de roomies **registrar y dividir sus gastos compartidos**, conocer en todo momento el **saldo de cada integrante** y **saldar sus deudas con el menor número razonable de transferencias**.

OBJETIVOS ESPECÍFICOS

Lo que se va a construir

1. Diseñar las estructuras de datos (`struct` y arreglos) para usuarios, grupos, categorías, gastos, divisiones y liquidaciones.
2. Implementar el registro de gastos con división igual o personalizada, validando que la suma de las partes coincida con el total.
3. Calcular el saldo neto de cada integrante considerando gastos y liquidaciones.
4. Implementar un algoritmo greedy de simplificación de deudas con máximo $n - 1$ transferencias.
5. Generar reportes de gastos por categoría, por persona y por mes.
6. Guardar y recuperar la información en archivos.
7. Organizar el código en módulos `.c/.h` con paso por referencia mediante apuntadores.

Dos tipos de usuario



Roomie

Integrante de un grupo que comparte gastos. Usa el sistema día a día. Entra con su nombre y un PIN de 4 dígitos.

Puede: registrar gastos, consultar saldos, ver la simplificación de deudas, registrar liquidaciones y generar reportes.

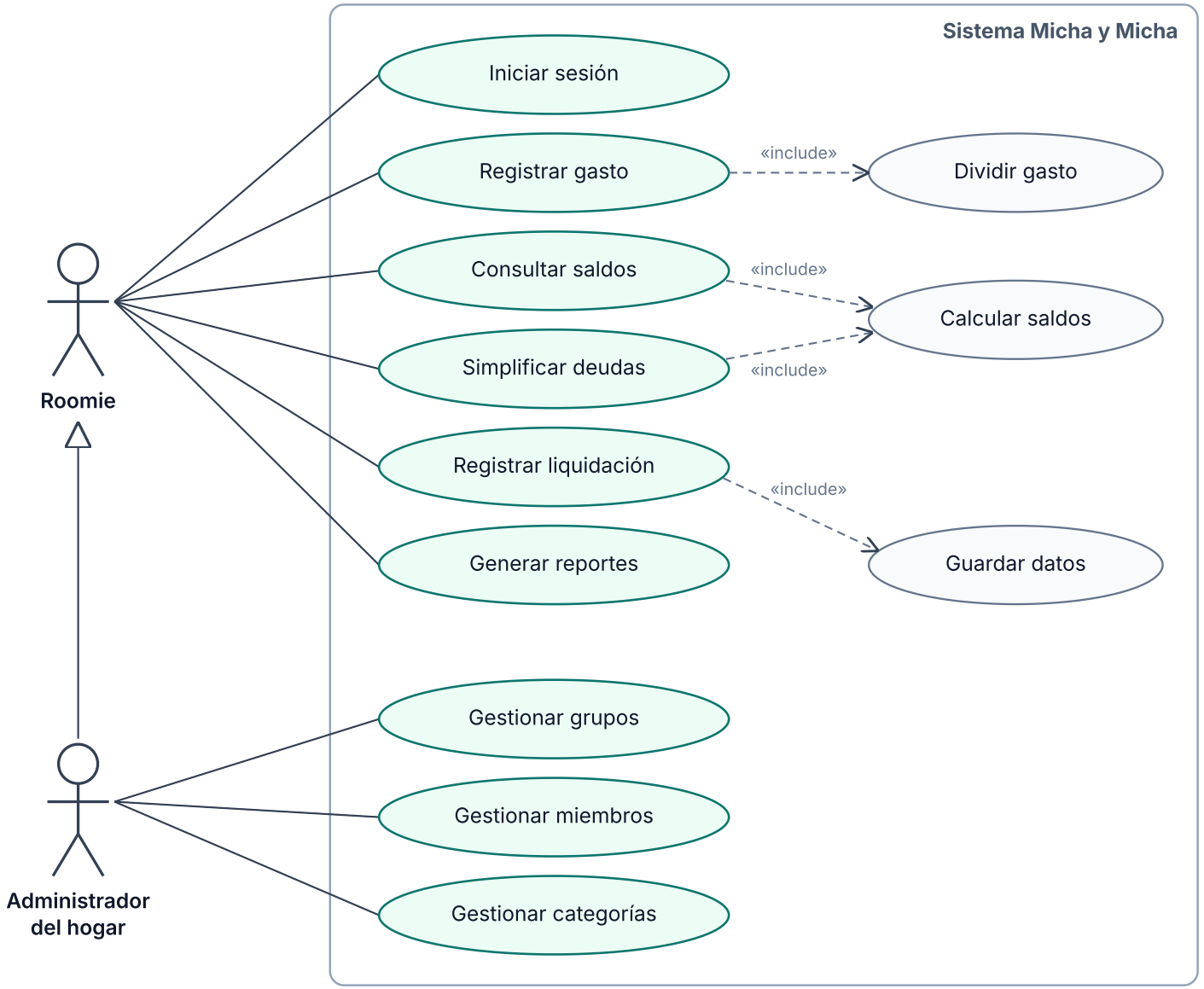


Administrador del hogar

Roomie con permisos extra; normalmente quien crea el grupo. Hereda todo lo que hace el Roomie.

Además puede: crear, editar y eliminar grupos; dar de alta y baja miembros; administrar categorías.

DIAGRAMA GENERAL DE CASOS DE USO



Roomie: 6 casos de uso del día a día.

Administrador: hereda los del Roomie (generalización) y agrega 3 de administración.

«include»: Dividir gasto, Calcular saldos y Guardar datos se ejecutan siempre dentro de otro caso de uso.

CASOS DE USO POR ACTOR

Funcionalidad general del sistema

CASO DE USO	ACTOR	QUÉ HACE
Iniciar sesión	Roomie / Admin	Selecciona usuario e ingresa PIN
Registrar gasto	Roomie	Captura monto, categoría, pagador y participantes; divide el gasto
Consultar saldos	Roomie	Muestra quién debe y a quién le deben
Simplificar deudas	Roomie	Sugiere máximo $n - 1$ transferencias
Registrar liquidación	Roomie	Anota un pago entre dos miembros
Generar reportes	Roomie	Historial, totales por categoría, persona y mes
Gestionar grupos / miembros / categorías	Administrador	Altas, bajas y cambios de la configuración